Lodge Billing Lab

ICT261 Lecture 3 | JavaFX Practical Lab | Mulungushi University

Complete Project Guide

Version 1.3







At a glance

Item	Detail
Project	Guest billing desk for a lodge
Course	ICT261 Java/JavaFX practical lab
Stack	Java 21, JavaFX 21.0.12, Gradle, Java Modules
Version	1.3 (v1.1 plus the cocoa redesign)

Rates and discount

Room	Per night
Standard	K 350.00
Deluxe	K 550.00
Family	K 750.00
Discount	2% off the room charge from 3 nights

How to use this guide

1. Read sections 1 and 2 for what the app does and the billing rules.
2. Follow section 3 to run it.
3. Use sections 5 to 12 to understand the code, theory first.
4. Finish with sections 14 and 17 to test and prepare for questions.

Contents

1. Project at a glance . . . 3
2. Billing rules . . . 4
3. Setting up and running . . . 5
4. Project structure and build files . . . 7
5. JavaFX concepts used in this project . . . 9
6. Main.java: the big picture . . . 10
7. Login and logout . . . 12
8. The billing logic . . . 13
9. Live summary and saving bills . . . 14
10. The saved bills table . . . 15
11. Dialogs and receipts . . . 16
12. Styling with CSS . . . 16
13. What changed since version 1.1 . . . 19
14. Testing . . . 20
15. Java and OOP concepts demonstrated . . . 21
16. Limitations and ideas for next steps . . . 21
17. Questions you may be asked (viva practice) . . . 22
18. Quick reference . . . 23

1. Project at a glance

Lodge Billing Lab is a JavaFX desktop application for a lodge front desk. A user logs in, enters a guest, a room type, the number of nights, any extras and the amount paid, and the app works out the bill, applies a discount for longer stays, shows the outstanding balance, and keeps a list of saved bills with printable-style receipts.

What the application does

- **Login and logout** with a hard-coded demo account and a confirmation dialog on logout.
- **Billing form**: guest name, room type, nights, extras, amount paid, notes.
- **Live bill summary** that recalculates as you type (room charge, discount, extras, total, paid, balance).
- **Saved bills table** with a Paid / Owing / Overpaid status, plus a totals strip (count, total billed, outstanding).
- **Receipts** for any saved bill, opened from a button or by double-clicking a row.
- **Design**: cocoa-and-tan theme with serif titles, round photos and a geometric accent.
- **Housekeeping**: New guest, Delete bill (with confirmation), File and Help menus, Ctrl+N and Ctrl+S shortcuts.

Technology

Technology	Version	Role in the project
Java	21	Language and runtime (records, switch expressions and pattern matching are used)
JavaFX controls	21.0.12	The UI toolkit: Stage, Scene, layout panes, TableView, dialogs, CSS
Gradle + org.openjfx.javafxplugin	0.1.0	Builds the project, downloads JavaFX, runs the app
Java Module System	-	module-info.java declares what the app needs (javafx.controls)
CSS (JavaFX flavour)	-	styles.css holds the whole visual theme
Bundled fonts and photos	-	Poppins and Lora fonts and three round photos, loaded from the resources folder

Demo account and room rates

Item	Value
Username	`admin`
Password	`admin123`
Standard room	K 350.00 per night
Deluxe room	K 550.00 per night
Family room	K 750.00 per night
Discount	2% off the room charge for 3 or more nights (extras are never discounted)
Tax	None in this lab

Data is not saved to disk
Bills live in memory only. They survive logging out and back in during the same run, but they disappear when you close the application. There is no database.

2. Billing rules

The rules are the heart of the lab. Everything else in the program exists to collect inputs and display these results.

Formulas

```
Room charge = Room price x Nights

If Nights >= 3:
    Discount = Room charge x 2%
else:
    Discount = 0

Total   = Room charge - Discount + Extras      (extras are NOT discounted)
Balance = Total - Amount paid
```

All money values are rounded to two decimal places at each step (room charge, discount, total, balance). This avoids floating-point noise such as `21.000000000000004`.

Balance status

Balance	Status	Meaning	Shown as
Greater than 0	Owing	Guest still has to pay	Amber pill, label 'Balance due'
Exactly 0	Paid	Fully settled	Green pill, label 'Balance'
Less than 0	Overpaid	Guest paid too much (credit)	Blue pill, label 'Overpaid by'

Worked example (from the README)

Guest John Phiri, Deluxe room, 3 nights, extras K200, paid K1000.

Step	Calculation	Result
Room charge	550 x 3	K 1,650.00
Discount (3+ nights)	1,650 x 2%	K 33.00
Discounted room charge	1,650 - 33	K 1,617.00
Total	1,617 + 200 extras	K 1,817.00
Balance	1,817 - 1,000 paid	K 817.00 (Owing)

More examples to test with

Room	Nights	Extras	Paid	Room charge	Discount	Total	Balance / status
Standard	1	0	0	350.00	0.00	350.00	350.00 Owing
Standard	2	0	700	700.00	0.00	700.00	0.00 Paid
Standard	3	0	1100	1,050.00	21.00	1,029.00	-71.00 Overpaid
Deluxe	3	200	1000	1,650.00	33.00	1,817.00	817.00 Owing
Deluxe	4	0	2156	2,200.00	44.00	2,156.00	0.00 Paid
Family	5	150	3000	3,750.00	75.00	3,825.00	825.00 Owing

3. Setting up and running

Requirements

- **JDK 21** (the Gradle toolchain asks for Java 21 specifically).
- **Gradle** or the Gradle Wrapper (`gradlew`).
- **Internet access** the first time, so Gradle can download JavaFX 21.0.12.

Run it

Open a terminal in the project folder (the one containing `build.gradle`).

```
# Windows (PowerShell)
gradlew run

# If the wrapper files do not exist yet
gradle wrapper
gradlew run

# Linux / macOS
./gradlew run
```

Run it in an IDE (IntelliJ IDEA)

1. Choose **Open** and select the `LodgeBillingLab` folder (or its build.gradle).
2. Let Gradle sync. In **Settings** > **Build Tools** > **Gradle** set the Gradle JVM to a JDK 21.
3. Open the Gradle tool window and run **Tasks** > **application** > **run**. Running the Main class directly also works in IntelliJ because Gradle supplies the JavaFX module path.

Troubleshooting

Symptom	Likely cause	Fix
Toolchain / 'Cannot find a Java installation' error	JDK 21 is not installed or not found	Install JDK 21 and re-run; check `java -version`
'gradlew' is not recognised	Wrapper files are not in the project	Run `gradle wrapper` once, then `gradlew run`
Module javafx.controls not found	Started with plain `java` instead of Gradle	Start with `gradlew run`
App runs but looks plain grey	`styles.css` was not found at run time	Confirm the file is in `src/main/resources/com/example/lodgebilling/`; run `gradlew clean run`
Fonts look generic or the round photos are missing	The fonts/ or images/ resource folders were not copied	Check both folders exist next to styles.css, then run `gradlew clean run`. The app still works without them
Strange characters in text	Source file read with the wrong encoding	build.gradle already forces UTF-8; make sure you did not remove that block
Compile error after editing	Typo or missing import	Read the first error line; it names the file, line and symbol

Build status of version 1.3
The version 1.3 interface was written without being compiled in the environment that produced it. Run `gradlew run` once and, if anything fails to compile or looks wrong, fix it using the table above or ask for a targeted correction with the exact error text.

4. Project structure and build files

```
LodgeBillingLab/
|-- build.gradle                build, dependencies, run settings
|-- settings.gradle             project name
|-- README.md                   documentation and testing checklist
`-- src/main/
    |-- java/
    |   |-- module-info.java     module declaration
    |   `-- com/example/lodgebilling/
    |       `-- Main.java        the whole application
    `-- resources/
        `-- com/example/lodgebilling/
            |-- styles.css       the visual theme
            |-- images/          photo1.png, photo2.png, photo3.png
            `-- fonts/           Poppins-Regular, Poppins-Bold, Lora-Variable
```

File	Purpose
build.gradle	Applies the application and JavaFX plugins, sets Java 21, UTF-8, resources folder, JavaFX modules and the main class
settings.gradle	Sets the project name to LodgeBillingLab
module-info.java	Declares the module, what it needs and what it exports
Main.java	All logic and UI: login, billing, table, receipt, dialogs, models
styles.css	Colours, fonts, spacing, buttons, cards, table, dialogs, receipt look
images/	Three round photos shown on the login and main screens
fonts/	Poppins (interface) and Lora (titles), with their open font licence note
README.md	Project description, billing rules, run instructions, testing checklist

build.gradle explained

Theory. Gradle reads build.gradle to learn which plugins to use and what to build. The JavaFX plugin knows how to download the right JavaFX jars for your operating system and pass them to Java when the app starts.

```
plugins {
    id 'application'                          // adds the 'run' task
    id 'org.openjfx.javafxplugin' version '0.1.0'   // downloads and wires JavaFX
}

java { toolchain { languageVersion = JavaLanguageVersion.of(21) } }

sourceSets {
    main { output.resourcesDir = java.destinationDirectory.get().asFile }
}

tasks.withType(JavaCompile).configureEach { options.encoding = 'UTF-8' }

javafx { version = '21.0.12'; modules = ['javafx.controls'] }

application {
    mainModule = 'com.example.lodgebilling'
    mainClass  = 'com.example.lodgebilling.Main'
}
```

- **toolchain**: forces JDK 21 regardless of what else is installed.
- **sourceSets block**: puts `styles.css` in the same folder as the compiled classes so the Java module can find it with `getResource`.
- **encoding**: compiles source as UTF-8 so special characters are not corrupted on Windows.
- **javafx.modules**: only `javafx.controls` is needed (it brings the graphics and base modules with it).
- **mainModule / mainClass**: tell `gradlew run` what to launch.

module-info.java explained

Theory. Since Java 9 a project can be a *module*. The module declares which other modules it requires and which of its packages other code may use.

```
module com.example.lodgebilling {
    requires javafx.controls;          // we use JavaFX controls
    exports com.example.lodgebilling;  // our package is visible to others
}
```

If you ever use `PropertyValueFactory` or FXML (both use reflection) you would add `opens com.example.lodgebilling to javafx.base;`. Versions 1.2 and later avoid reflection by using lambdas, so it is not needed.

5. JavaFX concepts used in this project

Read this section first if JavaFX is new to you. It gives the theory; later sections show the matching code.

Concept	What it means	Where it appears
Application	Base class for a JavaFX app. JavaFX calls `start(Stage)` after `launch()`	`Main extends Application`
Stage	The window	`stage` field, reused for login and main screens
Scene	The content shown in a Stage; holds a tree of nodes	`new Scene(root, w, h)`
Scene graph	Tree of nodes: panes contain controls, controls contain text	Every `create...` method returns part of the tree
Layout panes	Containers that position children: VBox (column), HBox (row), BorderPane (top/centre), GridPane (grid), StackPane (layers), ScrollPane	All screens
Controls	Interactive widgets: Button, TextField, ComboBox, TextArea, TableView, Label	Forms and tables
Event handling	Code that runs when the user acts, written as a lambda	`button.setOnAction(e -> saveBill())`
Properties and listeners	Observable values that notify you when they change	Text fields trigger the live summary
ObservableList	A list that notifies listeners when items are added or removed	`bills` feeds the TableView and the totals strip
Bindings	Connect one property to another so it updates automatically	Buttons disable when no row is selected
Style classes + CSS	Named labels on nodes that CSS rules target	`getStyleClass().add("card")`
Dialogs	Modal pop-up windows (Alert, Dialog)	Logout, delete, About, receipt

Application lifecycle

1. `main()` calls `launch(args)`.
2. JavaFX starts its UI thread and creates the Main object.
3. JavaFX calls `start(Stage)`. Here the app loads the stylesheet and shows the login screen.
4. Events (clicks, typing) run on the JavaFX Application Thread until the window closes or `Platform.exit()` is called.

6. Main.java: the big picture

Main.java is one file organised into labelled sections. Because the lab keeps everything in one class, the section banner comments are your map.

Section banner	What it contains
SCENE HELPERS	showScene, loadFonts, pattern, photoBubble, createPhotoColumn, styleDialog, infoDialog, confirm, field, autoHide, spacer, brandMark
LOGIN	showLoginScreen and authenticate
MAIN APPLICATION	showMainApplication: builds the menu bar and scrolling content, then shows it
TOP AREA	createMenuBar (File, Help) and createPageHeader (title, user chip, Log out chip)
PAGE CONTENT	createContent (page layout, pattern, photo column), createStatStrip, updateStats
FORM CARD	createFormCard, number-only input filters, select-on-focus
SIDE COLUMN	createSummaryCard (dark live summary) and createRateCard
SAVED BILLS TABLE	createBillsCard and the column helper
BILL CALCULATION	computeBill, parseNights, parseAmount, getRoomPrice, round2, money
SUMMARY	refreshSummary, showCalc, clearSummary, setStatus
SAVE BILL	saveBill
LOGOUT	logout
FORM CONTROLS AND TABLE ACTIONS	clearForm, deleteSelectedBill, viewReceipt
RECEIPT	showReceipt, receiptRow, dash, showAbout
MODEL	Calc and Bill records

Important fields

Field	Type	Purpose
stage	Stage	The single window; scenes are swapped inside it
stylesheet	String	URL of styles.css, or null if missing (app still runs, unstyled)
guestNameField … notesArea	TextField / ComboBox / TextArea	The form inputs
sum… labels, statusLabel	Label	The live summary card values and its message line
statCountValue, statBilledValue, statOwingValue	Label	The totals strip
billTable	TableView<Bill>	The saved bills table
bills	ObservableList<Bill>	In-memory store of saved bills
nextBillId	int	Counter that gives each bill a number (#001, #002…)

Screen layout as a tree

Login screen

```
HBox root
|-- StackPane brandPane      (triangle pattern, brand text, three round photos)
`-- StackPane formPane
    `-- VBox form            (title, fields, error message, Log in button, demo hint)
```

Main screen

```
BorderPane root
|-- top: MenuBar                       File, Help
`-- center: ScrollPane
    `-- StackPane layers
        |-- Pane pattern               nested triangles + diamond (behind everything)
        `-- HBox row
            |-- VBox left
            |   |-- HBox page header     title + subtitle, user chip, Log out chip
            |   |-- HBox stat strip      Bills saved | Total billed | Outstanding
            |   |-- HBox work area
            |   |   |-- VBox form card   GridPane of fields + Save / Calculate / New guest
            |   |   `-- VBox side column summary card (deep) + rates card
            |   `-- VBox bills card      title, View receipt, Delete, TableView
            `-- HBox photo column        vertical divider + three round photos
```

7. Login and logout

Theory. The app has one Stage. To move between login and billing it creates a new Scene and swaps it into the same Stage. Billing screens are only built after the credentials match, so the billing interface is not reachable without logging in.

Starting the app

```
@Override
public void start(Stage stage) {
    this.stage = stage;
    var url = Main.class.getResource("styles.css");
    stylesheet = (url == null) ? null : url.toExternalForm();
    showLoginScreen();
}
```

Swapping scenes

```
private void showScene(Scene scene, String title, double minWidth, double minHeight) {
    if (stylesheet != null) scene.getStylesheets().add(stylesheet);
    stage.setMinWidth(minWidth);
    stage.setMinHeight(minHeight);
    stage.setTitle(title);
    stage.setScene(scene);
    stage.sizeToScene();
    stage.centerOnScreen();
    stage.show();
}
```

Checking the credentials

```
if (USERNAME.equals(username) && PASSWORD.equals(password)) {
    showMainApplication(username);
} else {
    message.setText("Invalid username or password.");
    passwordField.clear();          // required by the lab
    passwordField.requestFocus();
}
```

- Pressing Enter in the username field moves to the password; Enter in the password field logs in.
- The error label is hidden when empty (`autoHide` binds visibility to the text).
- `USERNAME.equals(username)` is written constant-first so it can never throw a NullPointerException.

Logout

Both the header button and File > Log out call `logout()`. It shows a confirmation dialog; confirming calls `showLoginScreen()`. The bills list and bill counter belong to the Main object, so they are kept when you log in again.

> **Security note**
> The credentials are in the source code. That is acceptable for a teaching lab and unacceptable in production. A real system would store salted password hashes (for example BCrypt) in a database, use sessions and role checks.

8. The billing logic

Theory. Separate *reading input* from *calculating* from *displaying*. The code follows that: `computeBill()` reads and validates the fields and returns a plain result object; other methods only display it.

Result and bill records

A *record* is a compact immutable class: Java generates the constructor, getters (named like the fields), equals and toString.

```
private record Calc(String room, double rate, int nights, double roomCharge,
                    double discount, double extras, double total,
                    double paid, double balance) {
    String status() {
        if (balance > 0) return "Owing";
        return balance < 0 ? "Overpaid" : "Paid";
    }
}

private record Bill(int id, String guest, String notes, Calc calc, LocalDateTime issuedAt) { }
```

computeBill

```
private Calc computeBill() {
    String room = roomTypeComboBox.getValue();
    int nights = parseNights(nightsField.getText());
    double extras = parseAmount(extrasField.getText(), "Extras");
    double paid = parseAmount(amountPaidField.getText(), "Amount paid");

    double rate = getRoomPrice(room);
    double roomCharge = round2(rate * nights);
    double discount = nights >= DISCOUNT_MIN_NIGHTS
            ? round2(roomCharge * DISCOUNT_RATE) : 0;

    double total = round2(roomCharge - discount + extras);   // extras not discounted
    double balance = round2(total - paid);
    return new Calc(room, rate, nights, roomCharge, discount, extras, total, paid, balance);
}
```

Validation and exceptions

Problems are reported by throwing `IllegalArgumentException` with a readable message. The caller catches it and shows the message inline instead of opening an error dialog.

Input	Rule	Message
Nights	Required, whole number, greater than 0	Enter the number of nights. / Nights must be a whole number. / Nights must be greater than zero.

Input	Rule	Message
Extras, Amount paid	Empty counts as 0; must be a valid non-negative number	Extras must be a number. / … cannot be negative.
Guest name	Required when saving	Enter the guest name.

In addition, `TextFormatter` filters stop invalid typing: nights accepts up to 3 digits; money fields accept up to 9 digits and 2 decimals.

```
nightsField.setTextFormatter(new TextFormatter<String>(change ->
    change.getControlNewText().matches("\\d{0,3}") ? change : null));   // null = reject
```

Rounding and formatting helpers

```
private static double round2(double v) { return Math.round(v * 100.0) / 100.0; }

private static String money(double value) {
    String sign = value < -0.004 ? "\u2212" : "";
    return sign + "K " + MONEY.format(Math.abs(value));     // "K 1,817.00"
}
```

Why `double` is acceptable here: the lab uses small amounts and rounds each step. A production billing system should use `BigDecimal` (or integer ngwee) to avoid binary floating-point error.

9. Live summary and saving bills

How the summary stays in sync

Theory. Every TextField exposes a *text property*. Adding a ChangeListener runs your code each time the value changes. The form adds one to nights, extras, amount paid and the room ComboBox.

```
nightsField.textProperty().addListener((o, a, b) -> refreshSummary(false));
extrasField.textProperty().addListener((o, a, b) -> refreshSummary(false));
amountPaidField.textProperty().addListener((o, a, b) -> refreshSummary(false));
roomTypeComboBox.valueProperty().addListener((o, a, b) -> refreshSummary(false));
```

```
private void refreshSummary(boolean explicit) {
    try {
        Calc calc = computeBill();
        showCalc(calc);
        setStatus("Preview only. Use Save bill to keep this bill.", HINT);
    } catch (IllegalArgumentException e) {
        clearSummary();
        setStatus(e.getMessage(), explicit ? ERROR : HINT);   // soft hint while typing
    }
}
```

- **explicit = false** (typing): problems appear as a gentle hint.
- **explicit = true** (Calculate button): problems appear as a red error line.
- Status messages use three style classes: `status-hint`, `status-error`, `status-ok`.

Saving a bill

```
Calc calc = computeBill();
Bill bill = new Bill(nextBillId, guest, notesArea.getText().trim(), calc, LocalDateTime.now());
bills.add(bill);                                  // table and totals update automatically
billTable.getSelectionModel().select(bill);
billTable.scrollTo(bill);
setStatus(String.format("Saved bill #%03d for %s.", nextBillId, guest), OK);
nextBillId++;
```

Why the table updates by itself: the TableView was created with `bills` as its items list. Adding to an ObservableList notifies the table; the same list also has a ListChangeListener that calls `updateStats()`.

Totals strip

```
double billed = bills.stream().mapToDouble(b -> b.calc().total()).sum();
double owing  = bills.stream().mapToDouble(b -> Math.max(0, b.calc().balance())).sum();
```

Outstanding counts only bills with a positive balance; credit from overpaid bills is not subtracted.

10. The saved bills table

Theory. A TableView is made of TableColumns. Each column needs a *cell value factory* (what value to show for a row). Optionally a *cell factory* (how to draw the cell) and a *row factory* (how to draw or react to a row).

```
private static <T> TableColumn<Bill, T> column(String title, double width,
                                               Function<Bill, T> value) {
    TableColumn<Bill, T> column = new TableColumn<>(title);
    column.setCellValueFactory(cell ->
            new ReadOnlyObjectWrapper<>(value.apply(cell.getValue())));
    column.setPrefWidth(width);
    column.setSortable(false);
    return column;
}

TableColumn<Bill, String> guestColumn = column("Guest", 200, Bill::guest);
TableColumn<Bill, String> totalColumn = column("Total", 120, b -> money(b.calc().total()));
```

This helper replaces repeated `PropertyValueFactory` code and needs no reflection or `opens` declaration.

Custom cells

- **Balance column**: draws the amount and adds `balance-owing` (amber, bold) or `balance-clear`.
- **Status column**: shows a Label styled as a pill (`pill-paid`, `pill-owing`, `pill-overpaid`).
- **Row factory**: a double-click on a non-empty row opens the receipt.
- **Placeholder**: 'No bills yet…' is shown when the table is empty.

```
var noSelection = billTable.getSelectionModel().selectedItemProperty().isNull();
viewButton.disableProperty().bind(noSelection);     // buttons are disabled
deleteButton.disableProperty().bind(noSelection);   // until a row is selected
```

11. Dialogs and receipts

Dialog helpers

- `styleDialog(pane)`: attaches styles.css and the `app-dialog` style class so dialogs match the app.
- `infoDialog(...)`: builds the About dialog.
- `confirm(title, header, content, actionText, destructive)`: shows an action button and Cancel, returns true only if the action was chosen. Used by logout (normal) and delete (red, destructive).

```
Optional<ButtonType> result = alert.showAndWait();
return result.isPresent() && result.get() == action;
```

The receipt

`showReceipt(Bill)` builds a VBox 'sheet' (cream paper in the cocoa theme) and shows it in a `Dialog` with a Close button. It contains:

1. Brand mark, name, receipt number and issue date (`dd MMM yyyy, HH:mm`).
2. Guest and room with nights.
3. Room charge (with the nightly rate), discount, extras.
4. Total, amount paid, balance (coloured) and the status pill.
5. Notes (only if the user wrote any) and a thank-you footer.

Dashed separators are empty Regions with a dashed top border from CSS (`receipt-dash`). Rows come from one helper, `receiptRow(left, right, strong)`, so spacing stays consistent.

12. Styling with CSS

Theory. JavaFX CSS looks like web CSS but properties start with `-fx-` and the engine is smaller. Nodes get *style classes* in Java; the stylesheet decides how each class looks. A stylesheet attached to the Scene overrides the default Modena theme.

```
scene.getStylesheets().add(stylesheet);            // in showScene
card.getStyleClass().add("card");                  // in Java
.card { -fx-background-color: -card; -fx-background-radius: 16; }   /* in styles.css */
```

Colour palette (defined once on .root)

Token	Hex	Used for
-bg	#40281C	Window background (cocoa)
-bg-deep	#33201A	Login brand panel, deep summary card
-card / -card-line	#4F392D / #6B5446	Card fill and border
-field / -field-line	#5F4B40 / #7C6558	Inputs
-cream / -muted	#F3E6D8 / #C9B3A2	Main text / secondary text
-tan / -tan-btn / -tan-dark	#DCC0AA / #DDBB9B / #3B2518	Accents and rings / primary button / text on tan
-pattern	#855F48	Triangle pattern (set in Main.java, pattern method)
-ok, -warn, -danger, -info	#9AE5AD, #E8C77A, #FFB4A8, #A9CDF2	Paid, Owing, errors and delete, Overpaid

Style class map

Style class	Where it is used
app-title, app-sub	Serif page title and the small course line under it
chip, chip-button	Pill-shaped user chip and Log out button in the page header
card, stat-strip, summary-card	Solid cards with soft shadow, the totals strip, the deep live summary
btn-primary / btn-secondary / btn-ghost / btn-danger	Tan primary button, glass secondary, text-only, red outline (Save, Calculate, New guest, Delete)
card-title, field-label, page-sub, msg-muted	Typography levels
pill, pill-paid, pill-owing, pill-overpaid	Status badges in table and receipt
login-brand, login-form-pane, login-error	Login screen parts
photo-divider	Thin vertical line beside the round photos
app-dialog, danger	Dialog styling; red action button for delete
receipt, receipt-dash, row-label, row-value, row-strong	Receipt on cream paper
brand-mark	Lodge silhouette drawn with an SVG-style path (no image file)

Common customisations

Goal	What to change
Change the main colour	Edit the `-bg`, `-card`, `-tan` and `-tan-btn` values at the top of `.root`
Change the font	Edit `-fx-font-family` in `.root` (interface) and the Lora lines (titles); add your font file to `fonts/` and list it in `FONT_FILES`
Change the round photos	Replace photo1.png to photo3.png in `images/` (square images work best)
Rounder or squarer cards	Edit `-fx-background-radius` and `-fx-border-radius` in `.card`
New button look	Add a rule such as `.btn-x { ... }` and call `button.getStyleClass().add("btn-x")`
Different rooms or prices	Edit the three price constants and the ComboBox items; add a case in `getRoomPrice` and a `rateRow`
Different discount	Edit `DISCOUNT_RATE` and `DISCOUNT_MIN_NIGHTS`, then update the 'Discount (2%)' label text

13. What changed since version 1.1

Area	Version 1.1	Version 1.3
Styling	Default grey JavaFX look, inline styles	styles.css cocoa-and-tan theme, bundled Lora and Poppins fonts, round photos, triangle pattern
Login	Centred plain form	Split panel with brand side, pattern, photos and inline error box
Header	Menu bar and toolbar	Menu bar plus page header with title, user chip and Log out chip
Result display	One line of text after pressing Calculate	Live summary card with full breakdown
Totals	None	Bills saved, total billed, outstanding
Table	Plain columns	Right-aligned amounts, status pills, empty-state message, double-click for receipt
Receipt	Plain alert; no discount, extras or notes	Receipt on cream paper with full breakdown, date and notes
Errors	Modal error dialogs	Inline messages under the summary
Input	Free text	Digits-only nights, numeric money fields, select-on-focus
Delete	Deleted immediately	Confirmation dialog first
Exit	System.exit(0)	Platform.exit()
Money	Unrounded, '1817.00'	Rounded per step, '1,817.00'
Data model	Bill stored text strings	Bill and Calc records hold real numbers
Shortcuts	None	Ctrl+N new guest, Ctrl+S save bill

Version 1.2 introduced the live summary, totals strip, receipt and inline validation; version 1.3 restyled everything in the cocoa theme. Unchanged on purpose: the billing rules, room prices, demo credentials, in-memory storage and the overall feature list.

14. Testing

Manual test cases

#	Action	Expected result
1	Start the app	Login screen appears with brand panel
2	Log in with admin / wrong	Red message 'Invalid username or password.'; password field cleared
3	Log in with admin / admin123	Billing screen opens
4	Type nights = 3 with Deluxe, extras 200, paid 1000	Summary: 1,650.00 / -33.00 / 200.00 / total 1,817.00 / balance due 817.00
5	Change nights to 2	Discount shows a dash; total drops accordingly
6	Clear nights	Summary shows dashes and the hint 'Enter the number of nights.'
7	Press Calculate with nights empty	Same message in red
8	Press Save bill with empty guest	'Enter the guest name.'
9	Save a valid bill	Row appears as #001, totals strip updates, status message in green
10	Save with paid equal to total	Status pill shows Paid, balance 0.00
11	Save with paid above total	Status Overpaid; receipt label 'Overpaid by'
12	Select row, View receipt (or double-click)	Receipt with breakdown, date and notes
13	Delete bill, Cancel	Nothing is removed
14	Delete bill, Delete	Row removed; totals strip decreases
15	Log out, Cancel / confirm	Stays / returns to login; bills still there after logging in again
16	Ctrl+N and Ctrl+S	Form clears / bill saves

Edge cases worth trying

- Nights = 1, 2 (no discount) and 3 (first discounted night count).
- Extras with a decimal (for example 12.50) and a very large amount paid.
- Typing letters in nights or money fields (they should be ignored).
- Long notes on the receipt (text should wrap).
- Resizing the window small (the page scrolls; the minimum is 1040 x 640).

15. Java and OOP concepts demonstrated

Concept	Where it shows up
Classes and objects	`Main`; objects such as Label, Button, Bill
Encapsulation / private fields	All UI fields are private; records are private nested types
Constructors	Bill and Calc (generated by records); `new Scene(...)`, `new Label(...)`
Getter methods	Record accessors such as `bill.guest()`, `calc.total()`
Inheritance	`Main extends Application`; anonymous TableCell subclasses
Methods and decomposition	Small methods: computeBill, showCalc, refreshSummary, receiptRow
Conditional statements	Discount rule, status, balance colouring
Switch expressions	`getRoomPrice`, balance labels
Exception handling	try/catch for NumberFormatException; IllegalArgumentException for validation
Collections and streams	ObservableList, `bills.stream().mapToDouble(...)`
Generics	`TableColumn<Bill, String>`, `Function<Bill, T>`
Lambdas and functional interfaces	Event handlers, listeners, cell value factories
Static members and constants	Price constants, formatters, helper methods
Records (modern Java)	Calc and Bill
Pattern matching	`control instanceof Region region`
JavaFX event handling and GUI	Buttons, menus, shortcuts, listeners, tables, dialogs
Java Modules	module-info.java

16. Limitations and ideas for next steps

Current limitations

- Bills are lost when the app closes (in-memory list).
- Login is hard-coded and plain text; there is a single user.
- Money uses `double`.
- Everything is in one class, so the billing rules cannot be unit-tested without the UI.
- No printing or export of receipts; no check-in or check-out dates; no payment history per bill.

Improvement ideas, from easy to advanced

Idea	How to approach it
Save bills between runs	Write bills to a CSV or JSON file on exit and load on start; later move to SQLite with JDBC
Extract the billing rules	Move room prices and calculation into a `BillingService` class; test it with JUnit using the table in section 2
Use BigDecimal	Replace double in Calc and parsing; format with the same DecimalFormat
Safer login	Store a salted hash (BCrypt), load users from a file or database, add roles
Export receipt	Create a PDF or print the receipt Node with JavaFX PrintJob
Search and filter	Wrap `bills` in a FilteredList driven by a search field
Check-in / check-out dates	Add two DatePickers and compute nights from the date range
Package the app	Use jlink and jpackage to produce an installer

17. Questions you may be asked (viva practice)

Q. Why does the app extend Application?
JavaFX requires an Application subclass. launch() creates the instance and calls start(Stage), which is the entry point for building the UI.

Q. What is the difference between Stage and Scene?
The Stage is the window; the Scene is the content inside it. The app keeps one Stage and swaps Scenes for login and billing.

Q. How is the discount applied and why are extras excluded?
If nights >= 3 the room charge is reduced by 2%. The rule says only the room charge is discounted, so extras are added after the discount.

Q. How does the summary update without pressing a button?
Listeners on the nights, extras, paid and room controls call refreshSummary(false), which recomputes the bill and updates the labels.

Q. Why use records for Calc and Bill?
They are immutable data carriers with generated constructors and accessors, which keeps the model short and prevents accidental changes.

Q. How does the TableView know when a bill is added?
Its items are an ObservableList. Adding to the list notifies the table, and a ListChangeListener refreshes the totals strip.

Q. Why is exception handling used for validation?
parseNights and parseAmount throw IllegalArgumentException with a readable message; the caller catches it and shows the message inline.

Q. What does module-info.java do?
It declares the module name, requires javafx.controls and exports the package. It makes dependencies explicit.

Q. What are the weaknesses of the login?
Credentials are hard-coded and unhashed, one user only, no lockout or sessions. It is demo-only.

Q. Why can the app lose data and how would you fix it?
Bills are in memory. Persist them to a file or database on save and load them at startup.

Q. Why round at every step?
Binary doubles cannot represent many decimals exactly; rounding to 2 places keeps amounts clean. BigDecimal would be the production choice.

Q. How does CSS styling reach the controls?
Java adds style classes; styles.css, attached to the Scene, defines how each class looks and overrides the default theme.

18. Quick reference

Item	Value
Run	`gradlew run` (Windows) `./gradlew run` (Linux/macOS)
Login	`admin` / `admin123`
Rates	Standard K350, Deluxe K550, Family K750 per night
Discount	2% of room charge when nights >= 3
Total	Room charge - discount + extras
Balance	Total - amount paid
Status	Balance > 0 Owing; = 0 Paid; < 0 Overpaid
Shortcuts	Ctrl+N new guest, Ctrl+S save bill
Main code file	src/main/java/com/example/lodgebilling/Main.java
Theme file	src/main/resources/com/example/lodgebilling/styles.css
Minimum window	1040 x 640 (login: 800 x 520)